

电子科技大学

计算机专业类课程

实验报

课程名称：计算机网络编程

学 院：计算机学院

专 业：人工智能

学生姓名

指导教师：向渝

日 期： 2025 年 12 月 14 日

电子科技大学

实验报告

实验二

一、实验名称：并发的面向连接的 ECHO 服务器软件的实现

二、实验学时：4

三、实验内容和目的：

掌握服务端软件的工作原理。

掌握并发的面向连接的 ECHO 服务器软件的编程实现方法。

四、实验原理：

使用指定编程工具编写并发的面向连接的 ECHO 服务器软件的源代码。

使用指定编程工具对代码进行编译和调试，生成执行程序。

在 PC 机上执行程序，检验程序能提供并发的 ECHO 服务的功能。

五、实验步骤：

1.总体设计思路

本实验采用 C 语言基于 Socket 编程接口与 POSIX 线程库 (pthread)，实现一个支持多客户端并发访问的 TCP ECHO 服务器。服务器整体采用**“主线程监听 + 子线程处理客户端通信”**的并发模型，其基本流程为：

创建监听套接字 → 绑定端口 → 开始监听 → 接受客户端连接 → 为每个客户端创建独立线程 → 在线程中完成 ECHO 服务 → 连接关闭并释放资源

通过该设计，可以保证多个客户端同时连接服务器时，彼此之间互不影响，提高服务器的并发处理能力。

2.创建服务器监听套接字

服务器程序首先调用 `socket()` 创建一个基于 IPv4 的 TCP 套接字,用于监听客户端连接请求。若套接字创建失败，程序立即输出错误信息并终止执行，避免服务器进入异常状态。

```
server_fd=socket(AF_INET,SOCK_STREAM,0);
if(server_fd<0){
    perror("Failed to create socket");
    exit(1);
}
```

3.设置套接字选项（地址复用）

为避免服务器异常退出后端口被占用的问题，程序使用 `setsockopt()` 设置 `SO_REUSEADDR` 选项，使服务器能够快速重新绑定同一端口。

```
int opt=1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("Failed setsockopt");
    close(server_fd);
    exit(1);
}
```

4.绑定端口并进入监听状态

服务器通过 `bind()` 函数将套接字与本地 IP 地址和指定端口号（8888）绑定，然后调用 `listen()` 进入监听状态，等待客户端连接请求。此时服务器处于被动打开状态，能够同时处理多个客户端的连接请求。

```
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);
if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    perror("Failed to bind");
    close(server_fd);
    exit(EXIT_FAILURE);
}
if (listen(server_fd, 10) < 0) {
    perror("Failed to listen");
    close(server_fd);
    exit(EXIT_FAILURE);
}
printf("ECHO server is working, listening port %d...\n", PORT);
```

5.接收客户端连接并创建工作线程

在主线程中，服务器通过 `accept()` 接收客户端的连接请求。每当有新的客户端连接时，服务器为该客户端动态分配一个 `client_info_t` 结构体，用于保存客户端套接字和地址信息。随后，服务器调用 `pthread_create()` 创建一个新的线程，并将客户端信息传递给该线程进行独立处理，从而实现并发服务。

```

while (1) {
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    if (client_fd < 0) {
        perror("Failed to accept");
        continue;
    }
    client_info_t *client = malloc(sizeof(client_info_t));
    client->socket_fd = client_fd;
    client->address = client_addr;
    if (pthread_create(&thread_id, NULL, handle_client, (void *)client) != 0) {
        perror("Failed to create thread");
        close(client_fd);
        free(client);
        continue;
    }
    pthread_detach(thread_id);
}
}

```

6.线程函数中实现 ECHO 服务逻辑

在线程处理函数 `handle_client()` 中，服务器完成以下操作：

获取客户端 IP 地址和端口号并输出连接提示信息；

```

char client_ip[INET_ADDRSTRLEN];
inet_ntop(AF_INET,&(client->address.sin_addr),client_ip,INET_ADDRSTRLEN);
printf("New client has connected: %s:%d\n",client_ip,ntohs(client->address.sin_port));

```

循环调用 `recv()` 接收客户端发送的数据；

```

while((bytes_read=recv(client->socket_fd,buffer,BUFFER_SIZE-1,0))>0){
    buffer[bytes_read]='\0';
    printf("Message from %s:%d: %s",client_ip,ntohs(client->address.sin_port),buffer);
}

```

将接收到的数据原样返回给客户端，实现 ECHO 功能；

```
send(client->socket_fd,buffer,bytes_read,0);
```

当客户端发送 `quit` 命令或断开连接时，结束通信。

```

if(strncmp(buffer,"quit",4)==0){
    printf("client %s:%d request to leave\n",client_ip,ntohs(client->address.sin_port));
    break;
}

```

通过为每个客户端分配独立线程，服务器能够同时为多个客户端提供 ECHO 服务。

6.线程资源回收与连接关闭

在客户端通信结束后，线程调用 `close()` 关闭客户端套接字，并释放为客户端分配的动态内存资源。同时，主线程通过 `pthread_detach()` 将子线程设置为分离状态，避免线程结束后产生资源泄漏。

```

close(client->socket_fd);
printf("Client %s:%d has left\n",client_ip,ntohs(client->address.sin_port));
free(client);
pthread_exit(NULL);
}

```

六、实验数据及结果分析：

1.服务器启动与监听状态分析

编译并运行并发 ECHO 服务器程序后，服务器成功创建监听套接字并绑定指定端口（8888），终端输出服务器已进入监听状态的提示信息，表明服务器初始化过程正确完成，能够正常接收客户端连接请求。

```
yar@2023080910002:~/computer_network$ ./echo_server  
ECHO server is working, listening port 8888...
```

2.多客户端并发连接情况分析

在服务器运行过程中，分别在多个终端窗口中启动 TCP 客户端程序，同时连接至同一台服务器。实验结果表明，服务器能够在短时间内连续接收多个客户端连接请求，并为每个客户端创建独立的处理线程。服务器终端中依次输出不同客户端的 IP 地址和端口号，说明服务器具备同时处理多个连接的能力。

```
yar@2023080910002:~/computer_network$ ./echo_server  
ECHO server is working, listening port 8888...  
New client has connected: 127.0.0.1:44094  
New client has connected: 127.0.0.1:35232
```

3.并发数据交互效果分析

在多个客户端同时向服务器发送消息的情况下，服务器能够并行接收各个客户端的数据，并将数据分别原样回显给对应的客户端。实验过程中，不同客户端发送的消息内容互不干扰，各客户端均能正确接收到属于自身连接的回显数据，未出现数据错乱、阻塞或丢失的现象。该实验结果表明，采用“每连接一线程”的并发模型能够有效实现 ECHO 服务的并发处理。

```
Message from 127.0.0.1:44094: Hello im 44094  
Message from 127.0.0.1:35232: Hello im 35232
```

4.客户端退出与服务器稳定性分析

当某一客户端发送 quit 命令或主动断开连接时，服务器端能够正确识别该客户端的退出请求，输出对应的离线提示信息，并释放该客户端占用的资源。同时，服务器仍然保持运行状态，可以继续为其他已连接或新连接的客户端提供 ECHO 服务，未出现服务器异常终止的情况。

```
Message from 127.0.0.1:44094: quit  
client 127.0.0.1:44094 request to leave  
Client 127.0.0.1:44094 has left  
Message from 127.0.0.1:35232: hello
```

6.并发处理能力总结分析

综合实验结果可以看出，该并发 ECHO 服务器能够：同时接收多个客户端连

接；为每个客户端提供独立、可靠的数据回显服务；在客户端频繁连接、通信与退出的情况下保持稳定运行。实验充分验证了基于 TCP 和 pthread 的多线程服务器模型在并发通信场景下的有效性和可行性。